

```

/**
 * server/services/shorts-prep-pipeline.ts
 *
 * Shorts Pre-Publication Pipeline
 *
 * Runs BEFORE any YouTube API call. Does ALL AI work so that when YouTube
 * quota resets at midnight Pacific, the publisher just reads ready rows
 * and POSTs – zero AI calls at upload time.
 *
 * Pipeline per clip:
 *   1. Load clip + source video metadata from DB
 *   2. AI: score clip moment + extract hook line
 *   3. AI: generate YouTube Short title (≤100 chars, hook-first)
 *   4. AI: generate SEO description (keyword-rich, CTA, hashtags)
 *   5. AI: generate tag set (≤500 chars total, ranked by search volume)
 *   6. AI: generate thumbnail concept (composition, text overlay, color grade)
 *   7. Write all metadata to DB → status: 'ready_to_upload'
 *
 * The publisher (quota-aware-publisher.ts) reads 'ready_to_upload' rows
 * and calls the YouTube API. It makes zero AI calls.
 */

```

```

import { callOpenAI }    from '../lib/openai.js';
import { callClaude }    from '../lib/claude.js';
import { createLogger }  from '../lib/logger.js';
import { setJitteredInterval } from '../lib/timer-utils.js';
import { assertTierCapacity } from '../lib/ai-semaphore.js';
import { storage }        from '../storage.js';

```

```

const log = createLogger('shorts-prep-pipeline');

```

```

// ——— Types ———

```

---

```

interface ClipRecord {
  id: number;
  videoId: number;
  sourceVideoTitle: string;
  sourceVideoDescription: string;
  sourceVideoTags: string[];
  clipStartSec: number;
  clipEndSec: number;
  clipFilePath: string;
  gameName: string;
  channelName: string;
  hookMomentDescription?: string;
}

```

```

export interface ShortsReadyPayload {

```

```

clipId: number;
// YouTube metadata – all pre-computed, no AI at upload time
title:          string;    // ≤100 chars
description:     string;    // SEO-rich, ≤5000 chars
tags:           string[];  // ≤500 chars total
categoryId:      string;    // '20' = Gaming
defaultLanguage: string;    // 'en'
// Thumbnail
thumbnailConcept: ThumbnailConcept;
thumbnailFilePath: string | null; // set after thumbnail is rendered
// Pipeline state
status:         'ready_to_upload';
preparedAt:     Date;
}

interface ThumbnailConcept {
  composition:  string; // e.g. "small player vs massive enemy, rule-of-thirds left"
  textOverlay:  string; // ≤4 words, e.g. "I SURVIVED THIS"
  colorGrade:   string; // e.g. "desaturated blue shadows, hot orange highlights"
  focalElement: string; // e.g. "enemy explosion filling top 60% of frame"
  mood:         string; // e.g. "tense, disbelief, survival"
  avoidElements: string; // e.g. "no HUD, no minimap, no crosshair visible"
}

// — Main prep function —————

export async function prepareShortForUpload(clip: ClipRecord): Promise<ShortsReadyPayload> {
  log.info(`[ShortsPrepPipeline] Starting prep for clip ${clip.id} (video ${clip.videoId})`);

  const durationSec = clip.clipEndSec - clip.clipStartSec;

  // Step 1 – Hook extraction + moment scoring
  assertTierCapacity('shorts_pipeline', 'shorts-prep');
  const hookAnalysis = await callOpenAI({
    tier: 'shorts_pipeline',
    messages: [
      {
        role: 'system',
        content:
          'You are a YouTube Shorts analyst for a no-commentary PS5 gaming channel. ' +
          'Identify the single most compelling hook moment in a clip and write a ' +
          'punchy one-sentence description of what happens. Be specific and visual.',
      },
      {
        role: 'user',
        content:
          `Game: ${clip.gameName}\n` +
          `Source video: "${clip.sourceVideoTitle}"\n` +
          `Clip duration: ${durationSec}s (${clip.clipStartSec}s-${clip.clipEndSec}s)\n` +
          `Source description: ${clip.sourceVideoDescription?.slice(0, 400) ?? 'none'}\n\n` +

```

```

    'In one sentence, describe the peak action moment in this clip. ' +
    'Be specific: what happens, what the stakes feel like, why it is visually striking.',
  },
],
maxTokens: 120,
});

```

```

const hookMoment = hookAnalysis.choices[0].message.content?.trim() ?? clip.sourceVideoTitle;
log.info(`[ShortsPrepPipeline] Clip ${clip.id} hook: "${hookMoment}"`);

```

```

// Step 2 – Title generation

```

```

assertTierCapacity('shorts_pipeline', 'shorts-prep');

```

```

const titleResult = await callOpenAI({
  tier: 'shorts_pipeline',
  messages: [
    {
      role: 'system',
      content:
        'You are a YouTube Shorts title writer for a no-commentary PS5 gaming channel. ' +
        'Write titles that are ≤100 characters, hook-first (emotion or curiosity gap in ' +
        'the first 3 words), no clickbait, no ALL CAPS, no emojis. ' +
        'The title should make someone stop scrolling. ' +
        'Respond with ONLY the title, no quotes, no explanation.',
    },
    {
      role: 'user',
      content:
        `Game: ${clip.gameName}\n` +
        `Hook moment: ${hookMoment}\n` +
        `Source title: ${clip.sourceVideoTitle}\n` +
        `Duration: ${durationSec} seconds\n\n` +
        'Write the YouTube Short title.',
    },
  ],
  maxTokens: 60,
});

```

```

const title = titleResult.choices[0].message.content?.trim().slice(0, 100) ??
clip.sourceVideoTitle;
log.info(`[ShortsPrepPipeline] Clip ${clip.id} title: "${title}"`);

```

```

// Step 3 – SEO description

```

```

assertTierCapacity('shorts_pipeline', 'shorts-prep');

```

```

const descResult = await callOpenAI({
  tier: 'shorts_pipeline',
  messages: [
    {
      role: 'system',
      content:
        'You are an SEO specialist for a no-commentary PS5 gaming YouTube channel. ' +

```

```

        'Write a YouTube Short description that: ' +
        '(1) opens with the most searchable keyword phrase for this game and moment, ' +
        '(2) describes what happens in the clip in 1-2 sentences, ' +
        '(3) includes a subscribe CTA, ' +
        '(4) ends with 3-5 relevant hashtags including #Shorts. ' +
        'Total length: 150-300 characters. No filler phrases.',
    },
    {
        role: 'user',
        content:
            `Game: ${clip.gameName}\n` +
            `Title: ${title}\n` +
            `Hook moment: ${hookMoment}\n` +
            `Channel: ${clip.channelName}\n\n` +
            'Write the YouTube Short description.',
    },
],
maxTokens: 200,
});

```

```

const description = descResult.choices[0].message.content?.trim().slice(0, 5000) ?? '';
log.info(`[ShortsPrepPipeline] Clip ${clip.id} description ready (${description.length}
chars)`);

```

// Step 4 – Tag set

```
assertTierCapacity('shorts_pipeline', 'shorts-prep');
```

```
const tagsResult = await callOpenAI({
```

```
    tier: 'shorts_pipeline',
```

```
    messages: [
```

```
        {
```

```
            role: 'system',
```

```
            content:
```

```
                'You are a YouTube SEO specialist. Generate a tag set for a YouTube Short. ' +
```

```
                'Rules: (1) 8-15 tags, (2) mix of broad game tags and specific moment tags, ' +
```

```
                '(3) total character count ≤500 across all tags, ' +
```

```
                '(4) ranked best-first by estimated search volume, ' +
```

```
                '(5) respond with ONLY a JSON array of strings, no markdown.',
```

```
        },
```

```
        {
```

```
            role: 'user',
```

```
            content:
```

```
                `Game: ${clip.gameName}\n` +
```

```
                `Title: ${title}\n` +
```

```
                `Hook moment: ${hookMoment}\n` +
```

```
                `Existing source tags: ${clip.sourceVideoTags?.slice(0, 10).join(', ') ?? 'none'}\n\n`
```

```

+

```

```
                'Generate the tag array.',
```

```
        },
    ],
maxTokens: 200,

```

```

});

let tags: string[] = [];
try {
  const raw = tagsResult.choices[0].message.content?.trim() ?? '[]';
  const cleaned = raw.replace(/```json|```/g, '').trim();
  const parsed = JSON.parse(cleaned) as string[];
  // Enforce ≤500 chars total
  let totalChars = 0;
  tags = parsed.filter(t => {
    totalChars += t.length + 1;
    return totalChars <= 500;
  });
} catch {
  log.warn(`[ShortsPrepPipeline] Clip ${clip.id} tag parse failed – using fallback tags`);
  tags = [clip.gameName, 'Gaming', 'PS5', 'Shorts', 'NoCommentary'];
}
log.info(`[ShortsPrepPipeline] Clip ${clip.id} tags: [${tags.join(', ')}]`);

// Step 5 – Thumbnail concept
// Uses Claude for visual composition reasoning (better spatial thinking)
assertTierCapacity('shorts_pipeline', 'shorts-prep');
const thumbResult = await callClaude({
  tier: 'shorts_pipeline',
  messages: [
    {
      role: 'user',
      content:
        'You design YouTube gaming thumbnails. Describe a thumbnail concept for this\n\n' +
        `Game: ${clip.gameName}\n` +
        `Title: ${title}\n` +
        `Hook moment: ${hookMoment}\n\n` +
        'Output ONLY valid JSON with these fields:\n' +
        '{\n' +
        '  "composition": "subject positioning and rule-of-thirds description",\n' +
        '  "textOverlay": "4 words max, ALL CAPS, high contrast",\n' +
        '  "colorGrade": "shadow and highlight color direction",\n' +
        '  "focalElement": "the single most eye-catching element and its frame position",\n' +
        '  "mood": "2-3 emotion words",\n' +
        '  "avoidElements": "UI elements to hide or crop out"\n' +
        '}\n\n' +
        'For gaming thumbnails: prefer destruction/explosion in upper frame, ' +
        'small player vs massive threat compositions, minimal HUD visible, ' +
        'desaturated backgrounds with one hot color accent on the focal element.',
    },
  ],
  maxTokens: 300,
});

```

```

let thumbnailConcept: ThumbnailConcept = {
  composition: 'center subject, rule-of-thirds enemy upper right',
  textOverlay: 'INSANE MOMENT',
  colorGrade: 'cool blue shadows, orange fire highlights',
  focalElement: 'explosion filling upper 50% of frame',
  mood: 'tense, disbelief, survival',
  avoidElements: 'no HUD, no minimap, no health bar',
};

try {
  const raw = thumbResult.content[0].type === 'text'
    ? thumbResult.content[0].text.trim()
    : '{}';
  const cleaned = raw.replace(/``json|``/g, '').trim();
  thumbnailConcept = { ...thumbnailConcept, ...JSON.parse(cleaned) };
} catch {
  log.warn(`[ShortsPrepPipeline] Clip ${clip.id} thumbnail parse failed – using defaults`);
}

log.info(`[ShortsPrepPipeline] Clip ${clip.id} thumbnail concept ready`);

// Step 6 – Persist to DB as ready_to_upload
const payload: ShortsReadyPayload = {
  clipId: clip.id,
  title,
  description,
  tags,
  categoryId: '20', // Gaming
  defaultLanguage: 'en',
  thumbnailConcept,
  thumbnailFilePath: null, // populated by thumbnail renderer if configured
  status: 'ready_to_upload',
  preparedAt: new Date(),
};

await storage.upsertShortsReadyPayload(clip.id, payload);
log.info(`[ShortsPrepPipeline] ✅ Clip ${clip.id} → ready_to_upload | title: "${title}"`);

return payload;
}

// — Batch runner —————

/**
 * Process all pre-encoded clips that don't yet have ready metadata.
 * Runs on a jittered interval so the pipeline continuously prepares
 * clips ahead of the daily quota reset.
 */
export async function runShortsPrepCycle(userId: string): Promise<void> {
  log.info(`[ShortsPrepPipeline] Starting prep cycle for user ${userId}`);

```

```

const pendingClips = await storage.getEncodedClipsWithoutReadyPayload(userId);
log.info(`[ShortsPrepPipeline] ${pendingClips.length} clips awaiting prep`);

for (const clip of pendingClips) {
  try {
    await prepareShortForUpload(clip);
    // Small delay between clips to avoid hammering the AI queue
    await new Promise(r => setTimeout(r, 3000));
  } catch (err: any) {
    if (err.message?.includes('AI queue full')) {
      log.warn(`[ShortsPrepPipeline] Queue full – pausing prep cycle, will retry next interval`);
      break; // Don't spin – back off and let the queue drain
    }
    log.error(`[ShortsPrepPipeline] Failed to prep clip ${clip.id}:`, err);
    // Continue to next clip on non-queue errors
  }
}

log.info(`[ShortsPrepPipeline] Prep cycle complete`);
}

// — Service lifecycle —————

let stopPrepCycle: (() => void) | null = null;

export function startShortsPrepPipeline(userId: string): void {
  if (stopPrepCycle) {
    log.warn('[ShortsPrepPipeline] Already running – skipping double-start');
    return;
  }

  log.info('[ShortsPrepPipeline] Starting – prep interval: ~20 min');

  stopPrepCycle = setJitteredInterval(
    () => runShortsPrepCycle(userId).catch(err =>
      log.error('[ShortsPrepPipeline] Cycle error:', err)
    ),
    20 * 60 * 1000, // 20 min base, ±20% jitter
  );
}

export function stopShortsPrepPipeline(): void {
  if (stopPrepCycle) {
    stopPrepCycle();
    stopPrepCycle = null;
    log.info('[ShortsPrepPipeline] Stopped');
  }
}

```

